

Design-Prinzipien

**... oder warum wir trotz etlichen Frameworks immer noch
Software Ingenieure brauchen**

Was ist Objektorientierung?



- Die reale Welt ist komplex
- Durch OO wird versucht, Zusammenhänge der realen Welt in die Software zu übertragen
- Objekte der realen Welt finden sich in der Software wieder
- Beziehungen zwischen Objekten können abgebildet werden
- Objekte haben Zustände und Verhalten

Was ist Objektorientierung?

Konzepte / Werkzeuge

- Klassen (auch abstrakt)
- Vererbung
- Polymorphie
- Kapselung
- Assoziation
- Aggregation
- Komposition

Was ist



- Gestaltung
- früher: Formgebung, Formgestaltung - äußere Form eines Gegenstands
- Gestaltungsleitsatz: „Form follows function“

Was ist Softwaredesign?

- Auch: Softwarekonstruktion
- Gegensatz zu „Design“: SW-Design ist **NICHT** die äußere Form einer Software, also UI/UX
- SW-Design definiert die Struktur einer Software
 - High-Level-Design: Softwarearchitektur
 - Intermediate Design: Abstraktion
 - Low-Level-Design: Implementierung

Wozu das Ganze?

- Das Softwaredesign unterstützt die nichtfunktionalen Anforderungen
- Hauptsächlich die Wartbarkeit und Erweiterbarkeit
- Ein gutes SW-Design erhöht die Lebensdauer einer Software
- Software stirbt nicht an Abnutzung - Software stirbt an Entropie

Wie entsteht Softwaredesign?

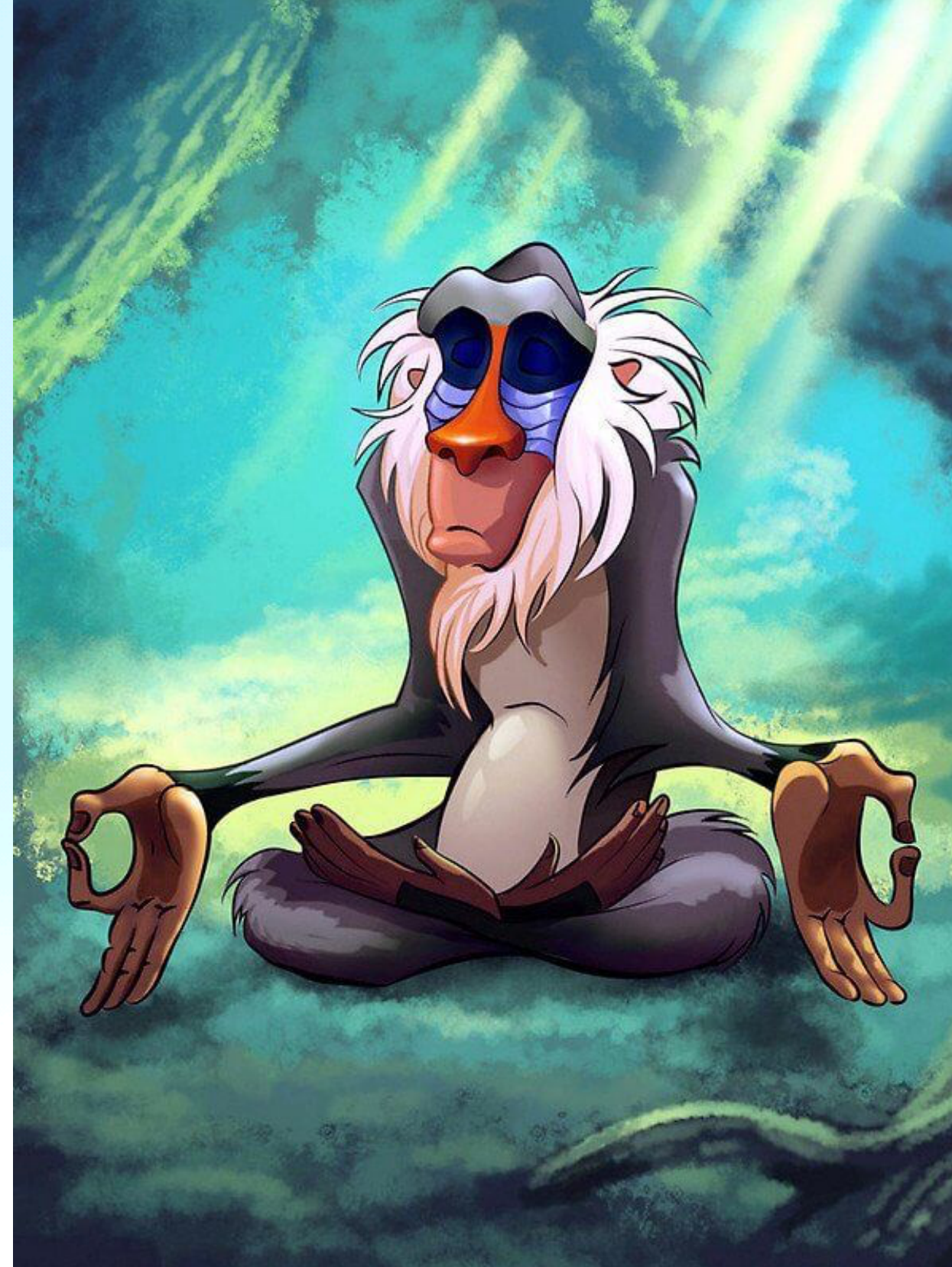
- NICHT:
 - Top-down
 - Vor dem Code (Big Upfront Design, waterfall model)
 - Durch Architekten und Designer
- Iterativ und inkrementell
- Während der Implementierung - Design ist ein Artefakt und nicht die Designphase

Woran erkennt man schlechtes SW-Design?

- Kleine Änderungen an einer Stelle führen zu einer Kaskade von Änderungen in weiten Teilen der Software
- Kleine Änderungen an einer Stelle führen zu Fehlern bei anderen Komponenten, die eigentlich gar nichts damit zu tun haben sollten
- Wiederverwendbarkeit ist nicht gegeben
- Viele, teils undurchsichtige Abhängigkeiten

SOLID Prinzipien

**Fünf Prinzipien zum Entwurf und Design von wartbaren
Softwaresystemen**



„Software sollte so komplex wie nötig, aber so einfach wie möglich sein“

Einführendes Beispiel

```
class Rectangle {  
  set width(value) {  
    this._width = value;  
  }  
  set height(value) {  
    this._height = value;  
  }  
  
  area() {  
    return this._width * this._height;  
  }  
}
```

```
function calculateArea(rect) {  
  rect.width = 5;  
  rect.height = 10;  
  
  return rect.area();  
}
```

```
class Square extends Rectangle {  
  set width(value) {  
    this._width = value;  
    this._height = value;  
  }  
  set height(value) {  
    this._width = value;  
    this._height = value;  
  }  
}  
  
const square = new Square();  
const result = calculateArea(square);  
console.log(result);
```

- Bereits bei wenigen Klassen / Funktionen können architekturelle/konzeptionelle Probleme auftreten
- Schlechtes Softwaredesign führt leicht zu schlecht wartbarem und schlecht erweiterbarem Code
- Gutes Softwaredesign ist so modelliert, dass Erweiterungen und Fehlerbehebungen einfach durchgeführt werden können

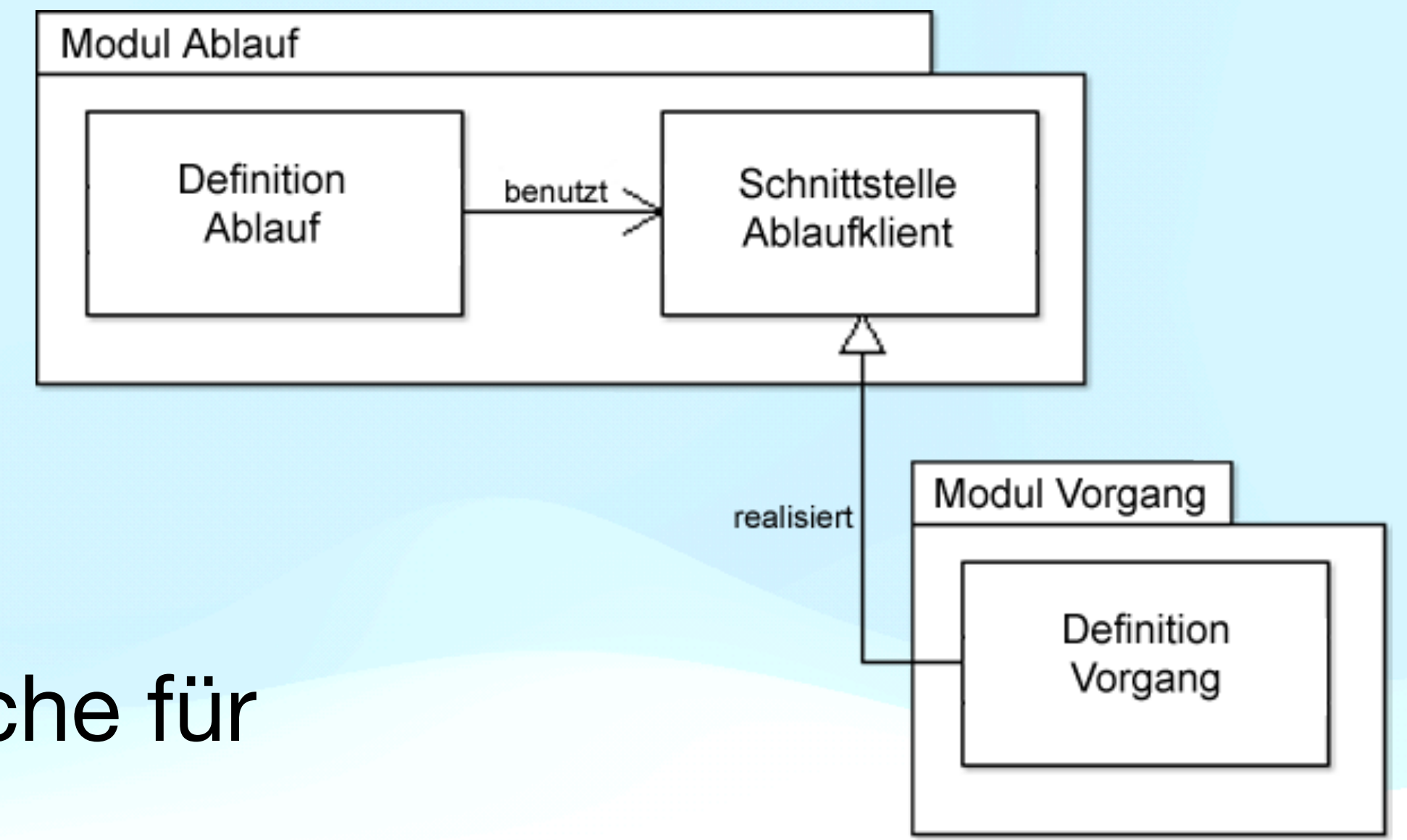
- Viele der SOLID Prinzipien behandeln Aspekte der Aggregation und Komposition von Klassen
- Hilfreich in der agilen Softwareentwicklung, da hier öfters Refactoring und Erweiterungen stattfinden
- Viele Techniken und Methoden der OOP basieren auf den SOLID Prinzipien

Na, was sind denn jetzt diese fünf Prinzipien?

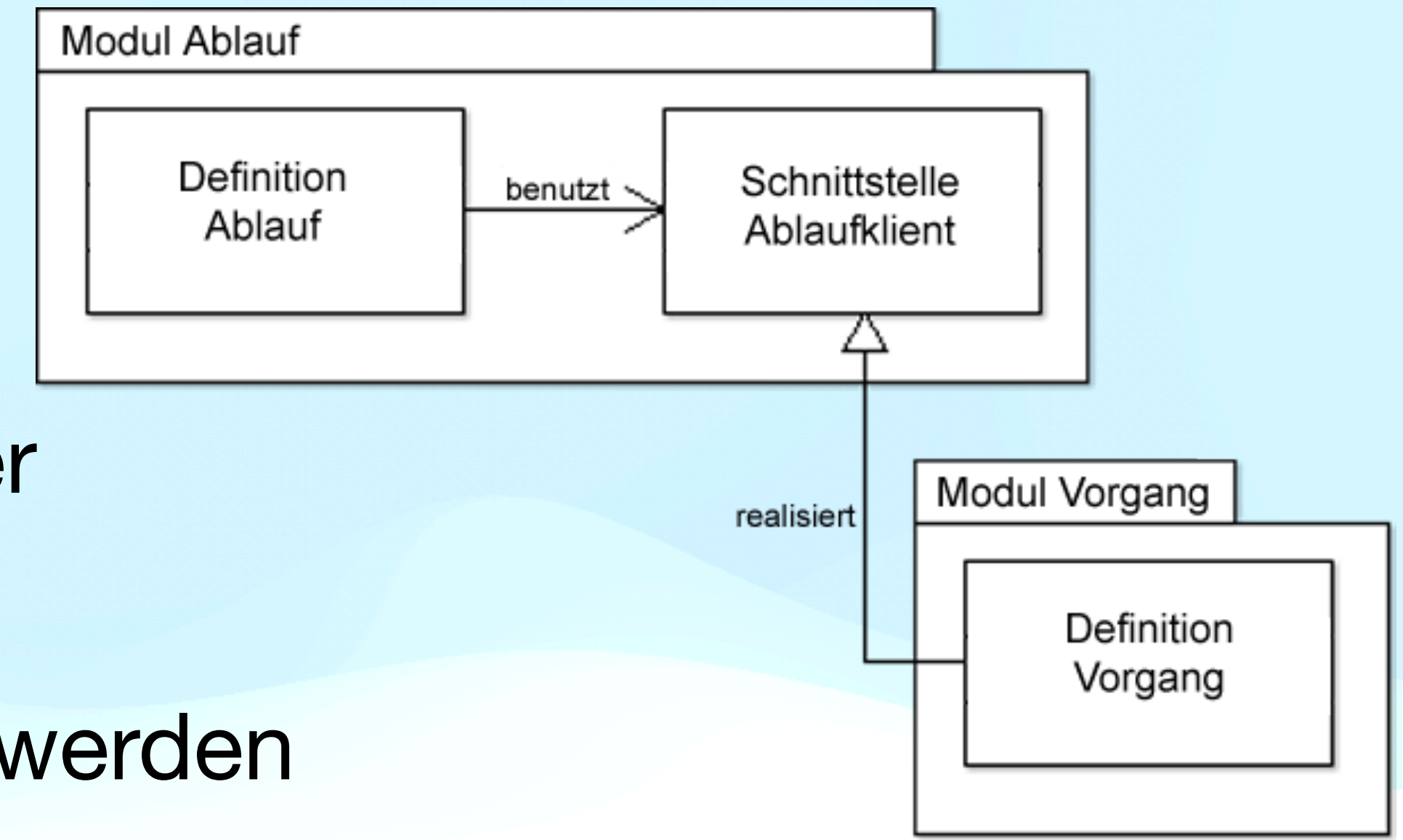
- Single Responsibility Principle (SRP)
- Open Closed Principle (OCP)
- Leskov Substitution Prinzip (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

Dependency Inversion Principle

- Kopplung zwischen Klassen ist häufige Ursache für schlechte Software
- Um schlechten Code auszutauschen, muss dieser möglichst isoliert betrachtet werden können
- Es muss verhindert werden, dass Klassen so miteinander verwoben sind, dass eine Änderung an einer einzelnen Klasse unmöglich wird
- ... Codebeispiele



- **High-level modules should not depend on low-level modules. Both should depend on abstractions.**



- Kopplungen zwischen Klassen werden über Abstraktionen aufgelöst
- Jede Klasse kann einzeln für sich getestet werden
- Der Servicenutzer definiert den Kontrakt. Dadurch ist die Interaktion genau auf diesen zugeschnitten
- **Abstractions should not depend on details. Details should depend on abstractions.**
- Durch die Abstraktion verschwindet das Interesse des Servicenutzers von der Implementierung des Serviceanbieters



Dependency Inversion Principle

Would you solder a lamp directly to the electrical wiring in a wall?

Single Responsibility Principle

- „Was ist die optimale Größe einer leicht wartbaren Klasse?“
- „A class should have only one reason to change“ - Robert C. Martin
- Werkzeug: Class-Responsibility-Collaboration

Class-Responsibility-Collaboration

Class	responsibility	Collaboration
Device	Abstrahierung der Anforderungen des Switch	-
Pump	• Geschäftslogik der Pumpe implementieren • Implementierung der Device Anforderungen	Device
Switch	Steuern eines angeschlossenen Devices	Device

Class-Responsibility-Collaboration

Class	responsibility	Collaboration
Device	Abstrahierung der Anforderungen des Switch	-
Pump	Geschäftslogik der Pumpe implementieren	-
PumpAdapter	Implementierung der Device Anforderungen	Device, Pump
Switch	Steuern eines angeschlossenen Devices	Device

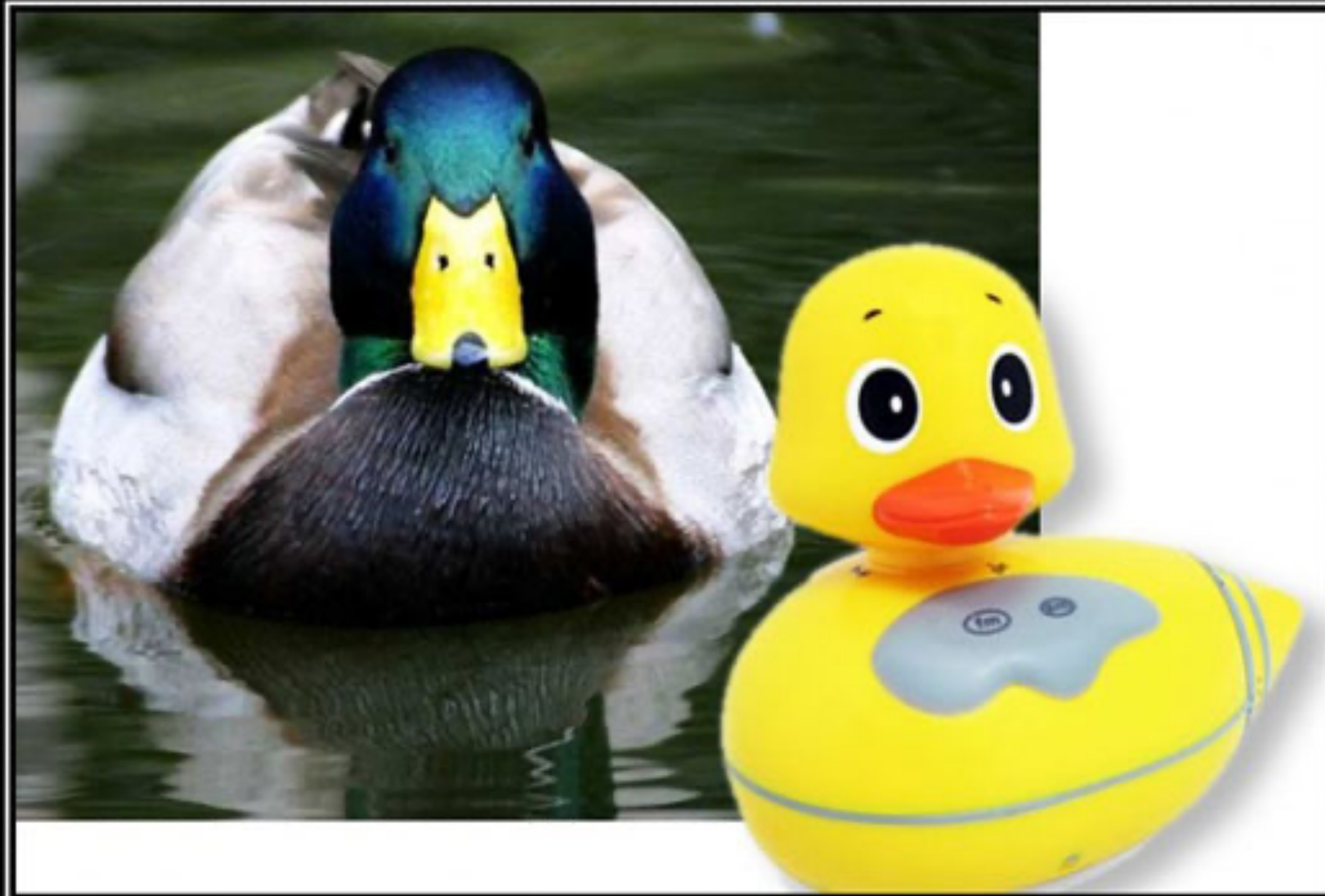


Single Responsibility Principle

Just because you *can* doesn't mean you *should*.

Liskov Substitution Principle

- Fortgeschrittenes Design-Prinzip - Impl. Von Spezialanforderungen
- *„Let $\Phi(x)$ be a property provable about objects x of type T . Then $\Phi(y)$ should be true for objects y of type S where S is a subtype of T .“*
- In anderen Worten: *Es muss möglich sein, eine Subklasse anstelle ihrer Superklasse zu verwenden.*
- Keine einfache „So geht’s Regel“
- Nur bei Vererbung praktikabel



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You
Probably Have The Wrong Abstraction

Interface Segregation Principle

- *„Clients should not be forced to depend on methods that they do not use.“*
- Interfaces / abstrakte Klassen dürfen nur die Methoden definieren, die auch von potentiell allen Subklassen implementiert werden.
- An der Verwendungsstelle kann dann z.B. ein Cast auf das zweite Interface gemacht werden
- Zu große Interfaces bringen oft Probleme mit sich. Die Wiederverwendbarkeit ist oft schlecht.
- ... Code



Interface Segregation Principle

You want me to plug this in *where*?

Open Closed Principle

- *„Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.“*
- Zwei Umsetzungswege:
 - Bertrand Meyer definiert 1988 die Umsetzung mittels Vererbung als geeignetes Werkzeug
 - Robert C. Martin nutzt Mitte der 90er Jahre nicht Vererbung sondern abstrakte Klassen zur Umsetzung
- Vererbung bringt Implementierungsprobleme / Aufwände mit sich: „privates“ müssen teilweise „protected“ werden, interner Code kann willkürlich von der Subklasse aufgerufen werden
- OCP ist nicht realistisch umsetzbar

Weitere Designprinzipien

- The Reuse/Release Equivalence Principle (REP)
- The Common Reuse Principle (CRP)
- The Common Closure Principle (CCP)
- The Acyclic Dependencies Principle (ADP)
- The Stable-Dependency Principle (SDP)
- The Stable-Abstractions Principle (SAP)
- Law of Demeter
- Dependency Injection

Mögliche Fragestellungen:

- Welches Problem soll gelöst werden?
- Wie wird ein Problem gelöst?
- Kritik an der Lösung
- Visualisierung (Code, UML, ...)

Dauer max. 10min